

passgen

Детерминированный генератор паролей

Полная документация для защиты проекта

1. Идея проекта и решаемая проблема

В современном мире у каждого человека десятки аккаунтов: почта, социальные сети, банки, работа, учеба. Для каждого требуется пароль. Использовать один пароль везде опасно — если злоумышленник узнает пароль от одного сервиса, он получит доступ ко всем остальным. Запоминать тридцать разных сложных паролей человеческая память не позволяет.

Менеджеры паролей хранят все пароли в одном зашифрованном месте, но создают единую точку отказа: если злоумышленник получит доступ к мастер-паролю, он получит все пароли сразу.

passgen предлагает принципиально другой подход — детерминированную генерацию. Вместо хранения паролей она каждый раз вычисляет их заново с помощью криптографического алгоритма HMAC-SHA256. Пользователю нужно запомнить всего одну фразу (сид), а пароль для любого сервиса восстанавливается по формуле: $\text{пароль} = \text{HMAC-SHA256}(\text{сид}, \text{сервис} + \text{длина} + \text{шаг})$.

Преимущества:

- Пароли не нужно хранить — они всегда восстанавливаются из сида и названия сервиса
- Нечего красть — на компьютере нет базы данных с паролями
- Пароль невозможно потерять — при знании сида он восстанавливается в любой момент
- Для каждого сервиса создается уникальный пароль
- Не требуется интернет — вся генерация происходит локально в браузере
- Программа бесплатна и имеет открытый исходный код

2. Структура программы

Вся программа находится в одном файле — index.html. Он содержит три компонента: HTML (разметка страницы), CSS (оформление) и JavaScript (логика работы). Никаких дополнительных файлов, библиотек или серверов не требуется.

2.1. HTML — разметка страницы

Интерфейс организован в виде карточек, расположенных вертикально:

1. Верхняя панель: заголовок "passgen", ссылки "Экспорт" и "Импорт".
2. Блок "Сид": поле ввода секретной фразы, место для ошибки.
3. Блок "Сервис": поле ввода названия, кнопка "список" с панелью сервисов.
4. Блок "Параметры": пресеты, длина со стрелками, 4 кружочка, расширенные настройки.
5. Кнопка "Создать".
6. Блок "Логин и пароль": поле логина с кнопкой, поле пароля, индикатор надежности.
7. Кнопка "Копировать".

8. Модальное окно импорта (скрыто, появляется при необходимости).

2.2. CSS — стили и оформление

Цветовая схема через CSS-переменные:

```
--bg: #f5f3ff — светло-фиолетовый фон
--card: #ffffff — белый фон карточек
--fg: #1a1a2e — темный текст
--border: #e5e7eb — цвет границ
--lilac: #a855f7 — фиолетовый акцент
--lilac2: #7c3aed — темно-фиолетовый для градиентов
--pink: #f472b6 — розовый (слабый пароль)
--crimson: #e11d48 — малиновый (средний пароль)
--error: #f43f5e — красный (ошибка)
```

Анимации: fadeUp (появление элементов), glow (пульсация при ошибке), slideDown/slideUp (раскрытие панелей), checkPop (переключение кружочков).

Карточки: белый фон, скругленные углы (12px), тень.

Кнопки: градиент, скругленные углы (11px), при нажатии scale(0.97).

2.3. JavaScript — логика программы

На JavaScript реализованы: генерация паролей через Web Crypto API, сохранение и восстановление настроек для каждого сервиса, экспорт и импорт в JSON, обработка событий. Web Crypto API — встроенная в браузер криптографическая библиотека, доступная в Chrome, Firefox, Edge, Safari.

3. Криптографический алгоритм

Объяснение того, как из сида и названия сервиса получается пароль.

3.1. Что такое HMAC-SHA256

HMAC-SHA256 — международный криптографический стандарт (RFC 2104). Используется в банковских системах, блокчейне, TLS/SSL, защите Wi-Fi. Алгоритм берет ключ (сид) и сообщение (сервис+длина+шаг) и вычисляет 256 бит (32 байта) данных.

Два важнейших свойства: результат невозможно предсказать без знания ключа; для одинаковых ключа и сообщения результат всегда одинаковый (детерминированность).

3.2. Пошаговая схема генерации

1. Сид превращается в байты через TextEncoder. Из байтов создается ключ HMAC через crypto.subtle.importKey.
2. Формируется сообщение: сервис + длина (как строка, затем байты).
3. Запускается цикл. К сообщению добавляется номер шага (0, 1, 2...). Вычисляется HMAC-SHA256: crypto.subtle.sign("HMAC", key, message).
4. Из 32 байтов каждый проверяется. Если байт < max, он превращается в символ из алфавита. Если нет — пропускается. Шаг увеличивается, цикл повторяется.
5. Если включена "Первая заглавная" — первый символ делается заглавным. Если "Последняя цифра" — последний символ заменяется цифрой (отдельный HMAC).

3.3. Rejection Sampling

256 (значений байта) не делится нацело на длину алфавита. Если бы программа брала байт % len(алфавит) для

всех байтов, первые символы встречались бы чаще последних (modulo bias). Это снижает криптостойкость.

Решение: $\text{max} = 256 - (256 \% \text{len}(\text{алфавит}))$. Байты $\geq \text{max}$ отбрасываются. Используются только байты $< \text{max}$ — для них распределение равномерно. Пример: алфавит из 50 символов: $\text{max} = 250$, байты 250-255 отбрасываются, каждый символ встречается ровно 5 раз.

3.4. Алфавиты наборов символов

Заглавные: ABCDEFGHJKLMNPQRSTUVWXYZ (нет I, O — путаются с 1, 0)

Строчные: abcdefghiklmnpqrstuvwxyz (нет j, l, o — путаются с цифрами)

Цифры: 0123456789

Спецсимволы: !@#\$%&*+=-.

По умолчанию: заглавные + строчные + спецсимволы, длина 10

3.5. Генерация логина

Логин через SHA-256 от сид + "login" + сервис + счетчик. Счетчик растет с каждым вызовом — каждый клик новый логин. Первые 6 байтов хеша — символы. Алфавит логина: "abcdefghijklmnopqrstuvwxyz23456789" (нет i,l,o — путаются с 1,0; нет 0,1 — путаются с буквами).

4. Интерфейс программы

Описание каждого элемента интерфейса.

4.1. Поле "Сид"

Самое важное поле. Сюда вводится секретная фраза — единственное, что нужно запомнить. Без сида невозможно восстановить пароль. При пустом поле — розовая пульсирующая граница, сообщение "Введите сид". Enter запускает генерацию.

4.2. Поле "Сервис" и список

Ввод названия сервиса. Кнопка "список" открывает панель с сервисами:

Google (синий G), GitHub (черный GH), VK (синий V), Яндекс (красный Ya),

Mail (синий @), iCloud (темно-синий i), Dropbox (синий D), Telegram (голубой T).

Клик по сервису заполняет поле и применяет настройки. Правый клик удаляет. Свой сервис можно ввести вручную — он добавится в список после генерации.

4.3. Параметры генерации

Пресеты: Свой, Минимальный (8, буквы+цифры), Стандартный (10, все кроме спецсимволов), Максимальный (16, все).

Длина: число от 1 до 99, стрелки для быстрой подстройки.

4 кружочка: заглавные, строчные, цифры, спецсимволы. Фиолетовый = включено.

Расширенные: "Первая заглавная" и "Последняя цифра" (скрыты по умолчанию).

4.4. Кнопка "Создать"

Запускает генерацию: проверка сида и чекбоксов, HMAC-SHA256, инверсия, отображение, копирование, добавление в список, сохранение. Фиолетовый градиент.

4.5. Результат

Логин (только чтение) + кнопка "Создать логин". Пароль (только чтение) — моноширинный, жирный, по центру.

Индикатор надежности: 3 полоски, цвет от розового до сиреневого, текстовая рекомендация.

4.6. Кнопка "Копировать"

Копирует пароль в буфер обмена через Clipboard API. При ошибке — `execCommand`.

4.7. Экспорт и импорт

Экспорт: скачивает JSON со всеми сервисами и настройками.

Импорт: загружает JSON, показывает окно выбора сервисов с чекбоксами.

5. Все функции JavaScript

Подробное описание каждой функции. Объяснения без кода, для устного ответа.

5.1. `init()` — подготовка программы

Запускается автоматически при загрузке. Очищает старые данные (`loadState`). Привязывает обработчики: ввод с клавиатуры снимает ошибку, Enter запускает генерацию, blur на сервисе запоминает выбор. Восстанавливает настройки сервиса или устанавливает стандартные. Рисует список сервисов.

5.2. `loadState()` — очистка для безопасности

Гарантирует чистый старт. Удаляет все из `localStorage`, обнуляет `perSvc` и `svcs`, сбрасывает длину на 10, очищает поле сервиса. Каждый запуск — с чистого листа.

5.3. `saveCfg()` — сохранение настроек

При нажатии "Создать" запоминает чекбоксы, инверсию и длину в `perSvc[сервис]`. Только в оперативной памяти — при закрытии исчезает.

5.4. `applyCfg()` — восстановление настроек

При выборе сервиса из списка восстанавливает его длину, чекбоксы, инверсию. Перерисовывает интерфейс.

5.5. `toggleInv()` — переключение инверсии

Клик по "Первая заглавная" или "Последняя цифра" переключает 0<->1. Обновляет кружок. Сохранение только при "Создать".

5.6. `applyPreset()` — пресеты

Минимальный (8, caps+lower+digits), Стандартный (10, caps+lower+digits), Максимальный (16, все 4). "Свой" — без действий.

5.7. `renderChk()` — чекбоксы

Создает 4 строки с кружками. Активный = фиолетовый. Клик переключает и перерисовывает.

5.8. `renderInv()` — кружки инверсии

Добавляет/убирает класс "on" на кружках cap и dig.

5.9. `adj()` — длина

Стрелки +/- 1. Не дает выйти за 1-99.

5.10. `toggleSvcPanel()` — список сервисов

Переключает класс `open` на панели. Показывает/скрывает с анимацией. Стрелка поворачивается.

5.11. renderSvcChips() — сборка списка

Сначала популярные (которых нет в сохраненных), потом все сохраненные сервисы.

5.12. chip() — кнопка сервиса

Популярные: цветной кружок с буквой. Свои: серый кружок с "?". Клик = заполнить поле. ПКМ = удалить.

5.13. gen() — генерация пароля (главная)

Самая важная функция. Запускается по "Создать" или Enter.

1. Проверка сида (иначе розовая ошибка)
2. Проверка чекбоксов (хотя бы один)
3. Сборка алфавита
4. hmacGen() — HMAC-SHA256
5. Первая заглавная (если включено)
6. Последняя цифра (если включено)
7. Отображение пароля
8. Индикатор надежности
9. Копирование в буфер
10. Добавление в список сервисов
11. Сохранение настроек

5.14. hmacGen() — криптографическое ядро

Асинхронная функция. Сид -> байты -> ключ HMAC. Сообщение: сервис+длина+шаг. Вычисляет HMAC (32 байта). Rejection sampling: байт < max -> символ. Повторяет, пока не наберется нужная длина.

5.15. genLogin() — логин

Проверяет сид и сервис. Увеличивает loginCounter. SHA-256 от сид+"login"+сервис+счетчик. Первые 6 байтов -> символы. Алфавит без i,l,o и 0,1. Каждый клик = новый логин.

5.16. updateStrength() — надежность

<10 или <3 наборов: слабый (розовый, 1 полоска). >=10 и >=3: средний (малиновый, 2). >=14 и 4: отличный (сиреневый, 3). Текстовая рекомендация.

5.17. cpy() — копирование

navigator.clipboard.writeText(), при ошибке execCommand. "Скопировано!" на 1.5с.

5.18. exportData() — экспорт JSON

Собирает сервисы с настройками, JSON с отступами, скачивает через ссылку.

5.19. importData() — импорт JSON

Читает файл через FileReader. Проверяет формат. Открывает окно выбора сервисов.

5.20. showImportModal() — окно импорта

Список сервисов с чекбоксами (все включены). Если пусто — ошибка.

5.21. confirmImport() — подтверждение импорта

Отмеченные сервисы: загружает настройки, добавляет в список. "Импорт завершен!"

5.22. closeImportModal() — закрытие

Убирает окно, очищает временные данные.

5.23. toggleAdvanced() — расширенные

Раскрывает/сворачивает панель инверсии.

5.24. getCfg() — настройки сервиса

Если нет — создает стандартные. Возвращает настройки.

5.25. onSvcChange() — смена сервиса

Обновляет curSvc, применяет настройки.

5.26. saveState() — загрузка

Пустая. Ранее сохраняла в localStorage. Оставлена для совместимости.

6. Обработка ошибок

1. Пустой сид — розовая подсветка, "Введите сид". При вводе исчезает.
2. Нет набора символов — "Выберите хотя бы один тип символов".
3. Длина вне 1-99 — принудительная граница.
4. Неверный файл импорта — сообщение об ошибке.
5. В файле нет сервисов — "Нет сервисов для импорта".
6. Clipboard недоступен — execCommand как запасной.

7. Экспорт и импорт (JSON)

```
{
  "svcs": ["google", "github"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

svcs — массив сервисов. perSvc — настройки для каждого: state (флаги символов), inv (инверсия), len (длина).

8. Безопасность

1. Локальность: все в браузере, данные не уходят в сеть.
2. Сессионный режим: настройки только в RAM, при перезагрузке все удаляется.
3. Стандартная криптография: HMAC-SHA256 (RFC 2104), используется в банках, TLS.
4. Rejection sampling: равномерное распределение символов.
5. Необратимость: зная пароль, нельзя восстановить сид.
6. Разные пароли для разных сервисов: название сервиса входит в HMAC.

Программа НЕ делает

Не хранит пароли (вычисляет заново). Не использует соль (сервис = соль). Не защищает от кейлоггеров. Не требует мастер-пароля (сид и есть мастер-пароль). Не создает резервных копий (сид храните отдельно).

9. Запуск программы

Файл index.html открывается любым способом:

1. Двойной клик по файлу -> браузер.
2. Ctrl+O в браузере -> выбрать файл.
3. HTTP-сервер (если нужно): `python3 -m http.server`, затем `localhost:8000`.

Требования:

- Любой современный браузер (Chrome, Firefox, Edge, Safari)
- Web Crypto API (встроен во все браузеры)
- Интернет не требуется
- Python не требуется (опционально для HTTP-сервера)